



**College of Computer and Information Sciences
Computer Science Department**

Project Tic-Tac-Toe Move Prediction

CSC462 - Machine Learning

Prepared by:

Anfal Alobeid 444200735

Rama Alfawzan 444201007

Maha Aldakhil 444202648

Batool Alyousef 444204725

Second Semester, 2026

Table of contents

I.	Introduction.....	3
II.	Data exploration	4
	2.1 Dataset Overview	4
	2.2 Label Distribution.....	4
	2.3 Sample Board Visualizations.....	5
III.	Methodology.....	5
	3.1 Baseline models	5
	3.1.1 Logistic Regression	5
	3.2 Advanced models	7
	3.2.1 Random Forest	7
	3.2.2 Gradient Boosting (XGBoost)	8
	3.2.3 Multi-Layer Perceptrons (MLP)	9
IV.	Results.....	11
V.	Conclusion.....	11
VI.	Appendix	12

I. Introduction

This project introduces four models for predicting the optimal next move in Tic-Tac-Toe given a board state, framed as a multi-class classification task with nine classes. The input features are the nine board positions (**p1–p9**), each taking a value of 0, 1, or -1 to represent an empty cell, the current player's mark, and the opponent's mark, respectively, along with a **turn** indicator (1 or -1) specifying which player is to move. The output is the predicted cell index (0–8) corresponding to the best next move.

Tic-Tac-Toe is a solved game for which the Minimax algorithm produces perfect play. However, this project simulates a real-world machine learning competition in which models must generalize to unseen data, testing whether supervised learning can recover the underlying game-theoretic structure from labeled examples alone, without being told the rules of the game. The core challenge is that each model has access only to board states and Minimax-labeled moves, and is ultimately evaluated on a hidden test set, ensuring that the success criterion is generalization rather than memorization.

Our approach begins with simple baselines (Logistic Regression) and progresses to more advanced models, namely Random Forest, XGBoost, and a Multi-Layer Perceptron (MLP). For each advanced model we apply targeted feature engineering and tune hyperparameters with an explicit focus on generalization, prioritizing configurations that produce a small train-validation gap rather than those that maximize training accuracy alone.

II. Data exploration

2.1 Dataset Overview

The dataset is partitioned into a training set of 3,164 examples and a validation set of 678 examples, with a hidden test set reserved for final evaluation. Preliminary inspection confirmed that the dataset is clean: no missing values or duplicate records were identified in either split, and all features fall within their expected ranges ($p1-p9 \in \{-1, 0, 1\}$, $turn \in \{-1, 1\}$, and $label_move_cell \in \{0, \dots, 8\}$). As the features are already represented on a bounded integer scale, no normalization or scaling was necessary.

2.2 Label Distribution

Analysis of the training labels revealed a notably imbalanced distribution of optimal moves, as illustrated in **Figure 1**. Cell 0 (top-left corner) emerged as the most frequent label with 765 occurrences, followed by cell 4 (center) with 489 occurrences and cell 2 (top-right corner) with 468 occurrences. The least frequent label is cell 7 (bottom-middle edge), appearing in only 117 training examples.



Figure 1: Distribution of optimal moves across the nine board cells (0–8) in the training set.

2.3 Sample Board Visualizations

To verify that the dataset labels are sensible and to better understand the prediction task, three random board states were drawn from the training set and visualized as 3×3 grids alongside their optimal-move labels **Figure 2**. In each case, the labeled move aligns with sound Tic-Tac-Toe strategy: the optimal cell either completes a winning line for the current player, blocks an immediate threat from the opponent, or claims a strategically valuable position such as the center. This qualitative inspection confirms that the Minimax-generated labels reflect optimal play and provides intuition for the patterns the model must learn — primarily tactical patterns involving rows, columns, and diagonals rather than properties of individual cells in isolation.

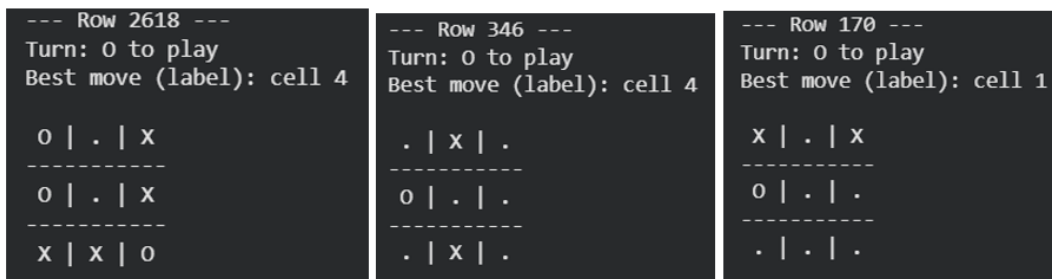


Figure 2: Three randomly sampled board states from the training set, showing the current player's turn and the optimal move label.

III. Methodology

3.1 Baseline models

For our baseline model, we chose Logistic Regression instead of options like Decision Trees or k-Nearest Neighbors (kNN). Decision Trees by nature can easily memorize the training data and overfit, which would give us an artificially high starting score and ruin the purpose of a baseline. On the other hand, kNN is less efficient for this type of discrete board data. Logistic Regression provides a simple, lightweight, linear starting point to compare against our advanced models later.

3.1.1 Logistic Regression

When we first ran the standard Logistic Regression model, we achieved a training accuracy of 25.06% and a validation accuracy of 21.98%. In Tic-Tac-Toe, choosing a move at random out of 9 possible empty squares would give about an 11% accuracy. This shows that our model was learning some basic patterns. However, because both scores were very low and close together, the model was clearly **underfitting**. The confusion matrix shown in **Figure 3**, confirmed this, as the predictions were scattered across all classes. The reason for this underfitting is that Tic-Tac-Toe relies on combinations of moves, but a basic logistic regression

model only evaluates one board square at a time. Making the model unable to understand the rules and patterns of the game.

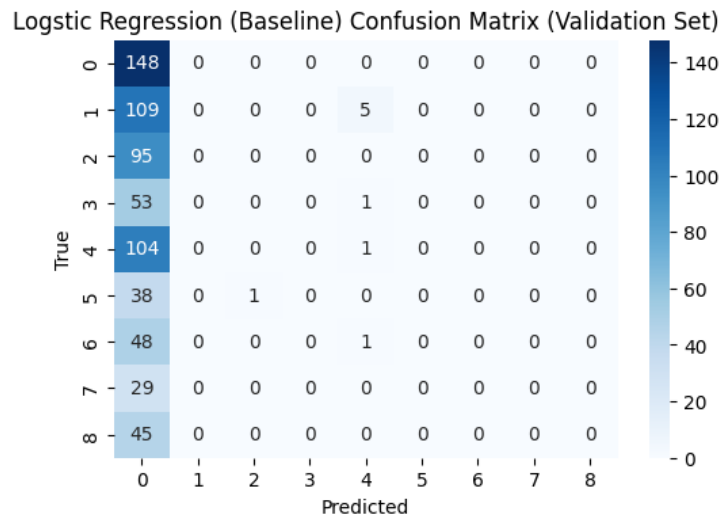


Figure 3: Confusion matrix for Logistic Regression baseline model, showing the scattered distribution of predictions across all classes due to underfitting.

To solve the underfitting, we applied Polynomial Features (degree=2) as a feature engineering technique. It multiplies the values of different cells together, creating new *interaction features*. This allows the model to process pairs of squares on the board, helping it recognize when a player is building a winning line, and produce better predictions. To prevent data leakage and deceptive results, we placed the transformation and model into a scikit-learn Pipeline. This ensured the feature engineering only learned from the training data, keeping our validation set completely pure.

With the new polynomial features, our training accuracy jumped to 85.78% and our validation accuracy reached 81.42%. The confusion matrix showing a strong diagonal line of correct predictions as shown in **Figure 4**. The small gap of about 4% between the training and validation scores proves that the model found a good fit and is generalizing the rules of the game, rather than just memorizing the data. This provides a solid baseline to measure our non-linear models against.

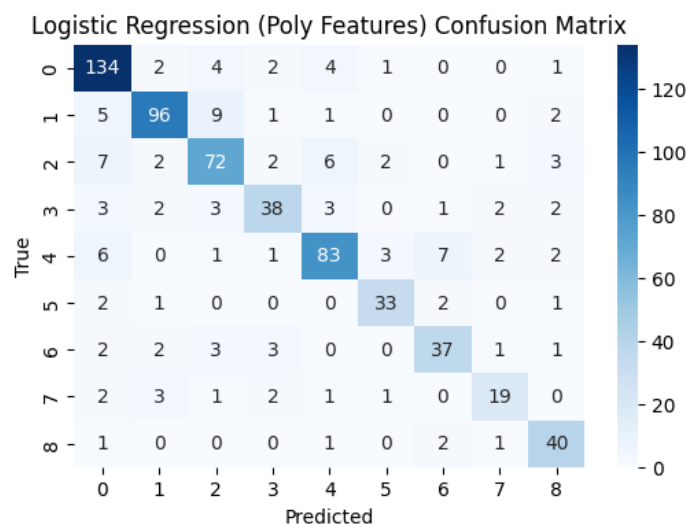


Figure 3: Confusion matrix for Logistic Regression baseline model with engineered polynomial features. The strong diagonal line of correct predictions demonstrates a high classification accuracy and correction of the underfitting.

3.2 Advanced models

3.2.1 Random Forest

Random Forest is an ensemble learning technique used in classification and regression tasks, built upon the foundation of decision trees. The model trains a large number of decision trees, each on a different bootstrap sample of the training data and considering only a random subset of features at each split as shown in the figure. For classification, predictions are aggregated through majority voting across all trees, which reduces the variance of any individual tree and improves the ensemble's generalization to unseen data.

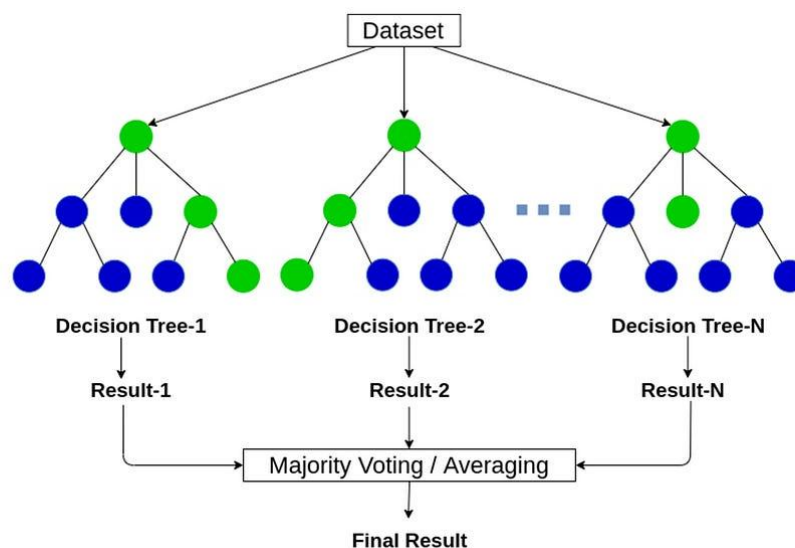


Figure 5: Random Forest Technique

We first trained a Random Forest on the raw 10 features (p1–p9, turn) to establish a controlled baseline that isolates the contribution of feature engineering. The model used the following hyperparameters: max_depth, min_samples_leaf, min_samples_split, max_features, max_samples, ccp_alpha, and class_weight. The depth and leaf-size constraints were chosen to prevent individual trees from memorizing the training data, while bootstrap subsampling and feature subsampling introduce diversity across trees, which is the central mechanism by which Random Forests achieve generalization. The class_weight='balanced' setting addresses the class imbalance observed in the dataset. With these settings, the model achieved 87.14% training accuracy and 75.22% validation accuracy, with a gap of 11.92 percentage points.

To improve performance, we introduced 27 engineered features designed to expose tactical patterns that raw cell values do not directly reveal. These features capture three categories of game logic: line-level threats (counts of two-in-a-row and one-in-a-row configurations for both players), per-cell tactical indicators (binary flags marking whether placing a mark at each cell would either win immediately or block an opponent's winning move), and positional control features (center occupancy, corner counts, and the number of empty cells as a game-phase indicator). With the same hyperparameters applied to this enriched feature set of 37 total features, the model achieved 82.71% training accuracy and 76.40% validation accuracy, with a gap of 6.31 percentage points. The reduction in the train-validation gap relative to the raw-features baseline confirms that game-aware feature engineering not only improves predictive performance but also enhances generalization.

3.2.2 Gradient Boosting (XGBoost)

XGBoost is an ensemble learning algorithm that builds a sequence of decision trees, where each new tree corrects the errors of the previous ones. This gradient-based approach is well-suited for tabular data with non-linear feature interactions, which matches the structure of the Tic-Tac-Toe move prediction task. The model was configured for multi-class classification across the nine possible board cells.

In Tic-Tac-Toe, optimal play depends on tactical patterns spanning multiple cells: forming winning lines and blocking opponent threats. To make these patterns explicit, two engineered features were added to the original board representation:

- **My two-in-a-row:** the number of winning lines on which the current player already has two marks and one empty cell, meaning a winning move is available.
- **Opponent two-in-a-row:** the number of winning lines on which the opponent has two marks and one empty cell, an immediate threat that must be blocked.

These features condense a non-linear pattern over the board into a single numeric signal, allowing the model to capture critical tactical decisions at shallower tree depths.

The model was trained for 200 boosting rounds with a learning rate of 0.1 and a maximum tree depth of 4. Shallow trees were chosen to limit complexity given the modest

training set size. To further control overfitting, L2 regularization was applied to the leaf weights, penalizing overly confident predictions and encouraging smoother decision boundaries. A baseline version without the engineered features and without regularization was also trained for comparison.

The baseline XGBoost achieved 91.59% training accuracy and 83.33% validation accuracy, with a gap of 8.26 percentage points indicating clear overfitting. Introducing the engineered features and applying L2 regularization produced the final model, which reached 90.30% training accuracy and 86.28% validation accuracy, reducing the gap to 4.01 percentage points. Although training accuracy decreased slightly, the gain on the validation set confirms that the improvement came from genuine generalization rather than memorization. **Figure 6** shows the per-round log-loss and accuracy curves for the final model.

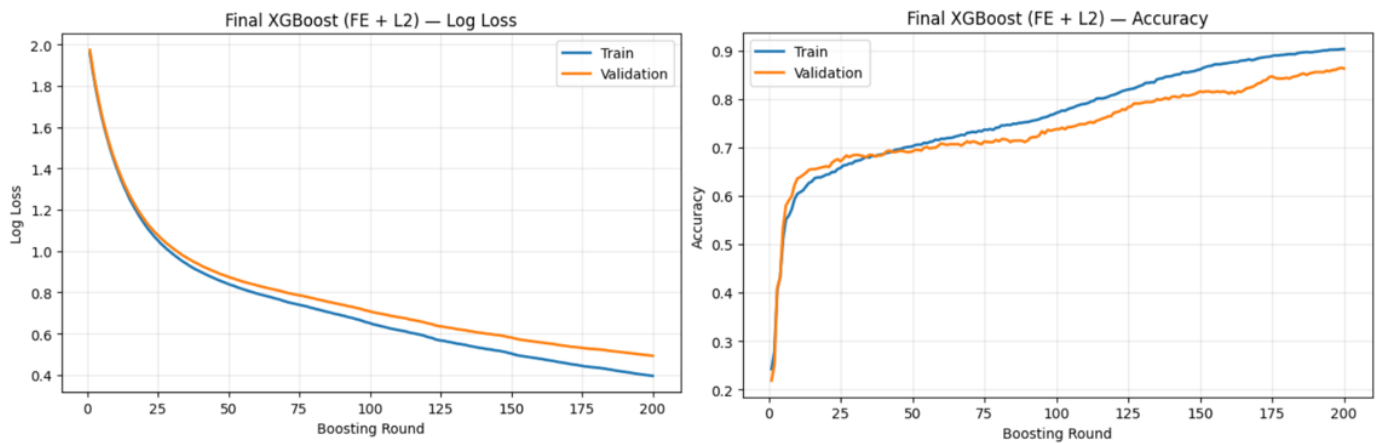


Figure 6: Training and validation curves for the final XGBoost model — log-loss (left) and accuracy (right)

3.2.3 Multi-Layer Perceptrons (MLP)

The chosen model is a Multi-Layer Perceptron (MLP), a type of feedforward neural network that learns non-linear relationships between inputs and outputs. The MLP was selected because the relationship between board states and optimal moves is not linear — for example, the strategic value of the center cell cannot be captured by a simple linear model.

The final architecture consists of two hidden layers with 64 and 32 neurons respectively, using ReLU as the activation function. L2 regularization ($\alpha=0.05$) was applied to reduce overfitting, and early stopping was enabled to halt training when validation performance stopped improving.

Feature Engineering

Seven strategic features were added on top of the original 10 features, bringing the total to 17:

- **my_two_in_row** — number of lines where the player has 2 marks and the third is empty (winning threat)
- **opp_two_in_row** — same for the opponent (blocking requirement)
- **my_fork** — 1 if the player has two or more simultaneous threats
- **opp_fork** — 1 if the opponent has two or more simultaneous threats
- **center_free** — 1 if the center cell is empty
- **my_corners** — number of corners owned by the player
- **empty_cells** — number of remaining empty cells, indicating the game stage

Evaluation Metrics

Two metrics were used to evaluate the models:

- **Accuracy** — percentage of correctly predicted moves on the validation set
- **Log-Loss** — measures the confidence of the predictions, penalizing wrong confident predictions
- **Accuracy Gap** — difference between training and validation accuracy, used to detect overfitting

...	=====	Original features : 10
	Baseline MLP (no feature engineering)	New features : 17
	=====	Added features : 7
	Train Accuracy : 0.8641	
	Val Accuracy : 0.7861	=====
	Accuracy Gap : 7.80 pp	MLP (with feature engineering)
		=====
	Train Log-Loss : 0.4114	Train Accuracy : 0.9314
	Val Log-Loss : 0.5913	Val Accuracy : 0.8584
	Loss Gap : 0.1800	Accuracy Gap : 7.30
	Training time : 1.14s	Train Log-Loss : 0.2365
		Val Log-Loss : 0.4352
		Loss Gap : 0.1986

Figure 7: Comparison of training and validation accuracy between baseline MLP and MLP with feature engineering.

As shown in Figure 7, the results demonstrate that feature engineering had a clear positive impact on model performance. Validation accuracy improved from 78.61% to 85.84%, and the overfitting gap decreased from 7.80% to 7.30%, indicating that the strategic features helped the model generalize better to unseen board states rather than memorizing the training data.

IV. Results

To evaluate our models, we focused on validation accuracy and the gap between training and validation performance. This ensured our models were genuinely generalizing to unseen board states rather than just memorizing the training data. Table 1 summarizes the final performance of all tested models.

Table 1: Model Performance Comparison

Model	Feature Engineering	Train Accuracy	Validation Accuracy	Train-Val Gap (generalization)
Standard Logistic Regression	No	25.06%	21.98%	3.08%
Random Forest	Yes (Strategic)	82.71%	76.40%	6.31%
Logistic Regression	Yes (Polynomial)	85.78%	81.42%	4.36%
MLP	Yes (polynomial)	93.14%	85.84%	7.30%
XGBoost	Yes + L2 Regularization	90.30%	86.28%	4.01%

We can see the standard Logistic Regression model severely underfits the data. However, by representing the board as interacting pairs (polynomial features) caused the validation accuracy to jump from roughly 22% to over 81%. This proved that for a game like Tic-Tac-Toe, how the data is represented is just as important as the algorithm used, thus we applied feature engineering to get the best results from each model.

When testing advanced architectures, Gradient Boosting (XGBoost) and the Multi-Layer Perceptron (MLP) performed the best, both scoring +85% on validation data. While the MLP showed strong predictive capabilities, XGBoost was our most successful model. By combining the engineered features with L2 regularization, XGBoost achieved the highest validation accuracy 86.28% while keeping the train-validation gap to just 4.01%. This small gap indicates a highly efficient model that successfully learned the underlying game strategies without overfitting. Ultimately, these results show that supervised learning can effectively capture the complex, non-linear rules of Tic-Tac-Toe from engineered labeled examples alone.

V. Conclusion

In this project we tried four different models to predict the best move in Tic-Tac-Toe and each one gave different results. Aside from choosing the right model for the right type of data, the most significant finding from our project is the impact of feature engineering. Adding things like two-in-a-row threats, fork detection, center control, and corner ownership gave the models more useful information about the game and helped them generalize better.

XGBoost was our best model overall with 86.28% validation accuracy and the lowest overfitting gap. MLP also did well with 85.84% which shows neural networks can work good for this kind of problem. If we had more time we would try adding more features, making the MLP bigger, and using grid search to find better settings for each model.

VI. Appendix

```
# **TASK 1**: Preparing the dataset
"""

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from google.colab import files

# Load CSVs
uploaded = files.upload()
train = pd.read_csv('train.csv')
val = pd.read_csv('val.csv')
#train = pd.read_csv('/content/sample_data/train.csv')
#val = pd.read_csv('/content/sample_data/val.csv')

# Quick look table
print("Training set shape:", train.shape)
print("Validation set shape:", val.shape)
print("\nFirst 10 rows of training data:")
train.head(10)

""""Checking the label distribution in the training set:*****

# Cell 2: Check label distribution
print("Label counts in training set:")
print(train['label_move_cell'].value_counts().sort_index())

# Visualize it
plt.figure(figsize=(8, 4))
train['label_move_cell'].value_counts().sort_index().plot(kind='bar', color='steelblue')
plt.title('Distribution of Best Moves in Training Set')
plt.xlabel('Cell Index (0-8)')
plt.ylabel('Number of Examples')
plt.xticks(rotation=0)
plt.grid(axis='y', alpha=0.3)
plt.show()

""""Vistlize 3 random boards from the Dataset :*****

def draw_board(row):

# Get the 9 cells
cells = row[['p1', 'p2', 'p3', 'p4', 'p5', 'p6', 'p7', 'p8', 'p9']].values
turn = row['turn']
label = row['label_move_cell']

# Convert numbers to symbols based on whose turn it is
# Current player (1) = X, Opponent (-1) = O
symbols = []
for v in cells:
    if v == 1:
        symbols.append('X' if turn == 1 else 'O')
    elif v == -1:
        symbols.append('O' if turn == 1 else 'X')
    else:
        symbols.append('.')

# Print as 3x3 grid
print(f"Turn: {'X' if turn == 1 else 'O'} to play")
print(f"Best move (label): cell {label}")
print()
print(f" {symbols[0]} | {symbols[1]} | {symbols[2]} ")
```

```

print("-----")
print(f" {symbols[3]} | {symbols[4]} | {symbols[5]} ")
print("-----")
print(f" {symbols[6]} | {symbols[7]} | {symbols[8]} ")
print()

# Pick 3 random rows and draw them
sample = train.sample(3, random_state=42)
for idx, row in sample.iterrows():
    print(f"--- Row {idx} ---")
    draw_board(row)

"""**Split features and labels for train and validation**"""

feature_cols = ['p1', 'p2', 'p3', 'p4', 'p5', 'p6', 'p7', 'p8', 'p9', 'turn']
target_col = 'label_move_cell'

X_train = train[feature_cols]
y_train = train[target_col]

X_val = val[feature_cols]
y_val = val[target_col]

print("X_train shape:", X_train.shape)
print("y_train shape:", y_train.shape)
print("X_val shape: ", X_val.shape)
print("y_val shape: ", y_val.shape)

print("\nFirst row of X_train:")
print(X_train.iloc[0].values)
print("Corresponding y_train:", y_train.iloc[0])

"""# **Logistic Regression (Baseline model)**

Without regularization
"""

import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix

# Train the Logistic Regression Baseline Model
lr_model = LogisticRegression(max_iter=1000, random_state=42)
lr_model.fit(X_train, y_train)

lr_train_preds = lr_model.predict(X_train)
lr_val_preds = lr_model.predict(X_val)

# Save the Standard Logistic Regression model
import joblib
from google.colab import files

joblib.dump(lr_model, 'standard_logistic_regression.pkl')
print("Standard model saved successfully!")
files.download('standard_logistic_regression.pkl')

# Showcase Accuracy
print("Logistic Regression (Baseline Model)")
print(f"Train Accuracy: {accuracy_score(y_train, lr_train_preds):.4f}")
print(f"Validation Accuracy: {accuracy_score(y_val, lr_val_preds):.4f}\n")

# Visualize with Confusion Matrix

```

```

cm = confusion_matrix(y_val, lr_val_preds)

plt.figure(figsize=(6, 4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.title('Logistic Regression (Baseline) Confusion Matrix (Validation Set)')
plt.xlabel('Predicted')
plt.ylabel('True')
plt.show()

"""With regularization (polynomial features)"""

from sklearn.preprocessing import PolynomialFeatures
from sklearn.pipeline import make_pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

# Added poly features of degree=2 to help the model see pairs of data, to help with
underfitting
# Create a pipeline that adds polynomial features first, then runs Logistic Regression to
prevent data leakage
poly_lr_model = make_pipeline(PolynomialFeatures(degree=2, interaction_only=True),
LogisticRegression(max_iter=3000, random_state=42))

poly_lr_model.fit(X_train, y_train)

poly_train_preds = poly_lr_model.predict(X_train)
poly_val_preds = poly_lr_model.predict(X_val)

print("Logistic Regression (Polynomial Features)")
print(f"Train Accuracy: {accuracy_score(y_train, poly_train_preds):.4f}")
print(f"Validation Accuracy: {accuracy_score(y_val, poly_val_preds):.4f}")

# Visualize with Confusion Matrix
poly_cm = confusion_matrix(y_val, poly_val_preds)

plt.figure(figsize=(6, 4))
sns.heatmap(poly_cm, annot=True, fmt='d', cmap='Blues')
plt.title('Logistic Regression (Poly Features) Confusion Matrix')
plt.xlabel('Predicted')
plt.ylabel('True')
plt.show()

# Save the Feature Engineered Pipeline model
import joblib
from google.colab import files

joblib.dump(poly_lr_model, 'poly_logistic_regression.pkl')
print("Feature Engineered model saved successfully!")
files.download('poly_logistic_regression.pkl')

"""# **XGBoost (Advance model)**

**( WITHOUT ) Feature Engineering**
"""

import xgboost as xgb
from sklearn.metrics import accuracy_score, log_loss
import time

# Baseline XGBoost (no feature engineering)
xgb_baseline = xgb.XGBClassifier(
n_estimators=200,
max_depth=4,

```

```

learning_rate=0.1,
min_child_weight=4,
objective='multi:softmax',
num_class=9,
random_state=42,
eval_metric=['mlogloss', 'merror'] # track both loss and error
)

# Train – pass eval_set so XGBoost records metrics every round
start = time.time()
xgb_baseline.fit(
X_train, y_train,
eval_set=[(X_train, y_train), (X_val, y_val)],
verbose=False
)
train_time = time.time() - start

# Save the per-round history so we can plot it later
baseline_history = xgb_baseline.evals_result()

# Predictions for accuracy (hard labels)
train_preds = xgb_baseline.predict(X_train)
val_preds = xgb_baseline.predict(X_val)

# Predictions for log-loss (probabilities)
train_proba = xgb_baseline.predict_proba(X_train)
val_proba = xgb_baseline.predict_proba(X_val)

train_acc = accuracy_score(y_train, train_preds)
val_acc = accuracy_score(y_val, val_preds)
train_loss = log_loss(y_train, train_proba)
val_loss = log_loss(y_val, val_proba)

print("Baseline XGBoost (no feature engineering)")
print(f"Training Accuracy: {train_acc:.4f} ({train_acc*100:.2f}%)")
print(f"Validation Accuracy: {val_acc:.4f} ({val_acc*100:.2f}%)")
print(f"Accuracy Gap: {(train_acc - val_acc)*100:.2f} pp")
print()
print(f"Training Log-Loss: {train_loss:.4f}")
print(f"Validation Log-Loss: {val_loss:.4f}")
print(f"Loss Gap: {val_loss - train_loss:.4f}")
print()
print(f"Training time: {train_time:.2f} seconds")

"""**Feature Engineering**"""

def add_strategic_features(df):
    """Add two-in-a-row features for both players (as suggested in project PDF)."""
    df = df.copy()

    # The 8 winning lines
    lines = [
        [0,1,2], [3,4,5], [6,7,8], # rows
        [0,3,6], [1,4,7], [2,5,8], # columns
        [0,4,8], [2,4,6] # diagonals
    ]

    board = df[['p1','p2','p3','p4','p5','p6','p7','p8','p9']].values

    my_two_in_row = []
    opp_two_in_row = []

    for b in board:
        my_2, opp_2 = 0, 0
        for line in lines:
            vals = [b[i] for i in line]

```

```

my_count = vals.count(1)
opp_count = vals.count(-1)
empty = vals.count(0)

if my_count == 2 and empty == 1: # I can win next move
my_2 += 1
if opp_count == 2 and empty == 1: # Opponent threatens to win
opp_2 += 1

my_two_in_row.append(my_2)
opp_two_in_row.append(opp_2)

df['my_two_in_row'] = my_two_in_row
df['opp_two_in_row'] = opp_two_in_row
return df

train_fe = add_strategic_features(train)
val_fe = add_strategic_features(val)

feature_cols_fe = ['p1', 'p2', 'p3', 'p4', 'p5', 'p6', 'p7', 'p8', 'p9', 'turn',
'my_two_in_row', 'opp_two_in_row']

X_train_fe = train_fe[feature_cols_fe]
y_train_fe = train_fe['label_move_cell']
X_val_fe = val_fe[feature_cols_fe]
y_val_fe = val_fe['label_move_cell']

print("Feature count:", X_train_fe.shape[1])
print("(10 original + 2 engineered = 12 features)")

def add_rich_features(df):
    """Richer feature engineering for Random Forest.
    Adds per-cell winning/blocking indicators + board-level features."""
    df = df.copy()

    lines = [
        [0,1,2], [3,4,5], [6,7,8], # rows
        [0,3,6], [1,4,7], [2,5,8], # columns
        [0,4,8], [2,4,6] # diagonals
    ]

    board = df[['p1', 'p2', 'p3', 'p4', 'p5', 'p6', 'p7', 'p8', 'p9']].values
    n = len(board)

    win_at = np.zeros((n, 9), dtype=int) # placing me here wins
    block_at = np.zeros((n, 9), dtype=int) # placing me here blocks opp

    my_two = np.zeros(n, dtype=int)
    opp_two = np.zeros(n, dtype=int)
    my_one = np.zeros(n, dtype=int)
    opp_one = np.zeros(n, dtype=int)

    for i in range(n):
        b = board[i]
        for line in lines:
            vals = [b[c] for c in line]
            mc, oc, ec = vals.count(1), vals.count(-1), vals.count(0)

            if mc == 2 and ec == 1:
                my_two[i] += 1
            for c in line:
                if b[c] == 0:
                    win_at[i, c] = 1
            if oc == 2 and ec == 1:
                opp_two[i] += 1
            for c in line:

```



```

if b[c] == 0:
    block_at[i, c] = 1
    if mc == 1 and ec == 2: my_one[i] += 1
    if oc == 1 and ec == 2: opp_one[i] += 1

df['my_two_in_row'] = my_two
df['opp_two_in_row'] = opp_two
df['my_one_in_row'] = my_one
df['opp_one_in_row'] = opp_one

for c in range(9):
    df[f'win_at_{c}'] = win_at[:, c]
    df[f'block_at_{c}'] = block_at[:, c]

df['center_mine'] = (board[:, 4] == 1).astype(int)
df['center_opp'] = (board[:, 4] == -1).astype(int)

corners = [0, 2, 6, 8]
df['corners_mine'] = sum((board[:, c] == 1).astype(int) for c in corners)
df['corners_opp'] = sum((board[:, c] == -1).astype(int) for c in corners)

df['empty_count'] = (board == 0).sum(axis=1)

return df

train_rf = add_rich_features(train)
val_rf = add_rich_features(val)

rf_feature_cols = [c for c in train_rf.columns if c != 'label_move_cell']
X_train_rf = train_rf[rf_feature_cols]
y_train_rf = train_rf['label_move_cell']
X_val_rf = val_rf[rf_feature_cols]
y_val_rf = val_rf['label_move_cell']

print(f"Rich FE feature count: {X_train_rf.shape[1]}")
print(f"(10 raw + 4 line counts + 18 per-cell + 5 board-level)")

"""**Train XGBoost on engineered features**

"""

import xgboost as xgb
from sklearn.metrics import accuracy_score, log_loss
import time

# XGBoost with Feature Engineering + L2 Regularization (Final Model)
xgb_fe = xgb.XGBClassifier(
    n_estimators=200,
    max_depth=4,
    learning_rate=0.1,
    min_child_weight=3,
    reg_lambda=11,
    objective='multi:softmax',
    num_class=9,
    random_state=42,
    eval_metric=['mlogloss', 'merror']
)

start = time.time()
xgb_fe.fit(
    X_train_fe, y_train_fe,
    eval_set=[(X_train_fe, y_train_fe), (X_val_fe, y_val_fe)],
    verbose=False)
train_time = time.time() - start

```

```

# Save the per round history
final_history = xgb_fe.evals_result()

train_preds = xgb_fe.predict(X_train_fe)
val_preds = xgb_fe.predict(X_val_fe)
train_proba = xgb_fe.predict_proba(X_train_fe)
val_proba = xgb_fe.predict_proba(X_val_fe)

train_acc = accuracy_score(y_train_fe, train_preds)
val_acc = accuracy_score(y_val_fe, val_preds)
train_loss = log_loss(y_train_fe, train_proba)
val_loss = log_loss(y_val_fe, val_proba)

print("Final XGBoost Model (FE + depth=4 + L2 regularization)")
print(f"Training Accuracy: {train_acc:.4f} ({train_acc*100:.2f}%)")
print(f"Validation Accuracy: {val_acc:.4f} ({val_acc*100:.2f}%)")
print(f"Accuracy Gap: {(train_acc - val_acc)*100:.2f} pp")
print()
print(f"Training Log-Loss: {train_loss:.4f}")
print(f"Validation Log-Loss: {val_loss:.4f}")
print(f"Loss Gap: {val_loss - train_loss:.4f}")
print()
print(f"Training time: {train_time:.2f} seconds")

"""**Plot**"""

import matplotlib.pyplot as plt
import numpy as np

# Extract the per round metrics from both models
# eval_set order was [train, val] validation_0 = train, validation_1 = val
b_train_loss = baseline_history['validation_0']['mlogloss']
b_val_loss = baseline_history['validation_1']['mlogloss']
b_train_acc = [1 - e for e in baseline_history['validation_0']['merror']]
b_val_acc = [1 - e for e in baseline_history['validation_1']['merror']]

f_train_loss = final_history['validation_0']['mlogloss']
f_val_loss = final_history['validation_1']['mlogloss']
f_train_acc = [1 - e for e in final_history['validation_0']['merror']]
f_val_acc = [1 - e for e in final_history['validation_1']['merror']]

rounds_b = np.arange(1, len(b_train_loss) + 1)
rounds_f = np.arange(1, len(f_train_loss) + 1)

fig, axes = plt.subplots(2, 2, figsize=(14, 9))

# Top-left: Loss for baseline
axes[0, 0].plot(rounds_b, b_train_loss, label='Train', linewidth=2)
axes[0, 0].plot(rounds_b, b_val_loss, label='Validation', linewidth=2)
axes[0, 0].set_title('Baseline XGBoost - Log Loss')
axes[0, 0].set_xlabel('Boosting Round')
axes[0, 0].set_ylabel('Log Loss')
axes[0, 0].legend()
axes[0, 0].grid(alpha=0.3)

# Top-right: Loss for final
axes[0, 1].plot(rounds_f, f_train_loss, label='Train', linewidth=2)
axes[0, 1].plot(rounds_f, f_val_loss, label='Validation', linewidth=2)
axes[0, 1].set_title('Final XGBoost (FE + L2) - Log Loss')
axes[0, 1].set_xlabel('Boosting Round')
axes[0, 1].set_ylabel('Log Loss')
axes[0, 1].legend()
axes[0, 1].grid(alpha=0.3)

```

```

# Bottom-left: Accuracy for baseline
axes[1, 0].plot(rounds_b, b_train_acc, label='Train', linewidth=2)
axes[1, 0].plot(rounds_b, b_val_acc, label='Validation', linewidth=2)
axes[1, 0].set_title('Baseline XGBoost - Accuracy')
axes[1, 0].set_xlabel('Boosting Round')
axes[1, 0].set_ylabel('Accuracy')
axes[1, 0].legend()
axes[1, 0].grid(alpha=0.3)

# Bottom-right: Accuracy for final
axes[1, 1].plot(rounds_f, f_train_acc, label='Train', linewidth=2)
axes[1, 1].plot(rounds_f, f_val_acc, label='Validation', linewidth=2)
axes[1, 1].set_title('Final XGBoost (FE + L2) - Accuracy')
axes[1, 1].set_xlabel('Boosting Round')
axes[1, 1].set_ylabel('Accuracy')
axes[1, 1].legend()
axes[1, 1].grid(alpha=0.3)

plt.tight_layout()
plt.show()

"""# **Random Forest (Advance model)**

**( WITHOUT ) Feature Engineering**
"""

# import needed libraries
from sklearn.ensemble import RandomForestClassifier
from sklearn.utils.class_weight import compute_class_weight
from sklearn.metrics import accuracy_score, log_loss
import numpy as np
import time

# class-weight setup
classes = np.arange(9) # nine classes

# calculate a weight for each class so rare classes get higher weights and vice versa
balanced_w = dict(zip(classes,
compute_class_weight('balanced', classes=classes, y=y_train)))

# set trees train
TOTAL_TREES = 200
CHECKPOINT = 25 # list of forest sizes
checkpoints = list(range(CHECKPOINT, TOTAL_TREES + 1, CHECKPOINT))

# the classifier definition
rf_baseline = RandomForestClassifier(
n_estimators=CHECKPOINT, # start training with 25 trees
max_depth=12, # no tree can grow deeper than 12 splits from root
min_samples_leaf=3, # every leaf contains 3 training samples or more
min_samples_split=10, # a node must have 10 samples or more
max_features=0.3, # at each split, only consider 30% of features for generalization
max_samples=0.7, # each tree trains on a random 70% bootstrap
ccp_alpha=0.001, # cost complexity pruning
class_weight=balanced_w, # apply the imbalance correction
warm_start=True, # add new trees to existing forest instead of rebuilding
random_state=42, # fixed seed for reproducibility
n_jobs=-1 # use all CPU cores in parallel
)

# to store metrics at each checkpoint
baseline_history = {
'n_trees': [],
'train_loss': [], 'val_loss': [],
'train_acc': [], 'val_acc': []
}

```

```

# training loop, starts by 25 tree then increment each time
for n in checkpoints:
    rf_baseline.n_estimators = n
    rf_baseline.fit(X_train, y_train)

# probability per class
train_proba = rf_baseline.predict_proba(X_train)
val_proba = rf_baseline.predict_proba(X_val)

# single most likely class
train_preds = rf_baseline.predict(X_train)
val_preds = rf_baseline.predict(X_val)

# record train and val log loss and accuracy at this forest
baseline_history['n_trees'].append(n)
baseline_history['train_loss'].append(log_loss(y_train, train_proba))
baseline_history['val_loss'].append(log_loss(y_val, val_proba))
baseline_history['train_acc'].append(accuracy_score(y_train, train_preds))
baseline_history['val_acc'].append(accuracy_score(y_val, val_preds))

# the final tree count (200) metrics
train_acc = baseline_history['train_acc'][-1]
val_acc = baseline_history['val_acc'][-1]
train_loss = baseline_history['train_loss'][-1]
val_loss = baseline_history['val_loss'][-1]

# printing results
print("=" * 55)
print("Random Forest WITHOUT Feature Engineering")
print("=" * 55)
print(f"Training Accuracy: {train_acc:.4f} ({train_acc*100:.2f}%)")
print(f"Validation Accuracy: {val_acc:.4f} ({val_acc*100:.2f}%)")
print(f"Accuracy Gap: {(train_acc - val_acc)*100:.2f} pp")
print(f"\nTraining Log-Loss: {train_loss:.4f}")
print(f"Validation Log-Loss: {val_loss:.4f}")
print(f"Loss Gap: {val_loss - train_loss:.4f}")
print("=" * 55)

"""**Feature Engineering and training**"""

from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, log_loss
import time

# set trees train
TOTAL_TREES = 200
CHECKPOINT = 25 # list of forest sizes
checkpoints = list(range(CHECKPOINT, TOTAL_TREES + 1, CHECKPOINT))

# the classifier definition
rf_fe = RandomForestClassifier(
    n_estimators=CHECKPOINT, # start training with 25 trees
    max_depth=12, # no tree can grow deeper than 12 splits from root
    min_samples_leaf=3, # every leaf contains 3 training samples or more
    min_samples_split=10, # a node must have 10 samples or more
    max_features=0.3, # at each split, only consider 30% of features for generalization
    max_samples=0.7, # each tree trains on a random 70% bootstrap
    ccp_alpha=0.001, # cost complexity pruning
    class_weight=balanced_w, # apply the imbalance correction
    warm_start=True, # add new trees to existing forest instead of rebuilding
    random_state=42, # fixed seed for reproducibility
    n_jobs=-1 # use all CPU cores in parallel
)

# to store metrics at each checkpoint

```

```

final_history = {
    'n_trees': [],
    'train_loss': [], 'val_loss': [],
    'train_acc': [], 'val_acc': []
}

# training loop, starts by 25 tree then increment each time
for n in checkpoints:
    rf_fe.n_estimators = n
    rf_fe.fit(X_train_rf, y_train_rf)

# probability per class
train_proba = rf_fe.predict_proba(X_train_rf)
val_proba = rf_fe.predict_proba(X_val_rf)

# single most likely class
train_preds = rf_fe.predict(X_train_rf)
val_preds = rf_fe.predict(X_val_rf)

# record train and val log loss and accuracy at this forest
final_history['n_trees'].append(n)
final_history['train_loss'].append(log_loss(y_train_rf, train_proba))
final_history['val_loss'].append(log_loss(y_val_rf, val_proba))
final_history['train_acc'].append(accuracy_score(y_train_rf, train_preds))
final_history['val_acc'].append(accuracy_score(y_val_rf, val_preds))

# the final tree count (200) metrics
train_acc = final_history['train_acc'][-1]
val_acc = final_history['val_acc'][-1]
train_loss = final_history['train_loss'][-1]
val_loss = final_history['val_loss'][-1]

# printing results
print("=" * 55)
print("Random Forest WITH Feature Engineering")
print("=" * 55)
print(f"Training Accuracy: {train_acc:.4f} ({train_acc*100:.2f}%)")
print(f"Validation Accuracy: {val_acc:.4f} ({val_acc*100:.2f}%)")
print(f"Accuracy Gap: {(train_acc - val_acc)*100:.2f} pp")
print(f"\nTraining Log-Loss: {train_loss:.4f}")
print(f"Validation Log-Loss: {val_loss:.4f}")
print(f"Loss Gap: {val_loss - train_loss:.4f}")
print("=" * 55)

"""**plot**"""

import matplotlib.pyplot as plt

b = baseline_history
f = final_history

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# plot a figure for training and val losses (without feature engineering)
axes[0].plot(b['n_trees'], b['train_loss'], label='training_loss')
axes[0].plot(b['n_trees'], b['val_loss'], label='val_loss', color='orange')
axes[0].set_title('Random Forest WITHOUT Feature Engineering - Loss')
axes[0].set_xlabel('Number of trees')
axes[0].set_ylabel('Log-loss')
axes[0].legend()
axes[0].grid(alpha=0.3)

# plot a figure for training and val losses (with feature engineering)
axes[1].plot(f['n_trees'], f['train_loss'], label='training_loss')
axes[1].plot(f['n_trees'], f['val_loss'], label='val_loss', color='orange')
axes[1].set_title('Random Forest WITH Feature Engineering - Loss')

```

```

axes[1].set_xlabel('Number of trees')
axes[1].set_ylabel('Log-loss')
axes[1].legend()
axes[1].grid(alpha=0.3)

plt.tight_layout()
plt.show()

import matplotlib.pyplot as plt

b = baseline_history
f = final_history

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# --- Left: RF without FE ---
axes[0].plot(b['n_trees'], b['train_acc'], label='training_acc')
axes[0].plot(b['n_trees'], b['val_acc'], label='val_acc', color='orange')
axes[0].set_title('Random Forest WITHOUT Feature Engineering – Accuracy')
axes[0].set_xlabel('Number of trees')
axes[0].set_ylabel('Accuracy')
axes[0].legend()
axes[0].grid(alpha=0.3)

# --- Right: RF with FE ---
axes[1].plot(f['n_trees'], f['train_acc'], label='training_acc')
axes[1].plot(f['n_trees'], f['val_acc'], label='val_acc', color='orange')
axes[1].set_title('Random Forest WITH Feature Engineering – Accuracy')
axes[1].set_xlabel('Number of trees')
axes[1].set_ylabel('Accuracy')
axes[1].legend()
axes[1].grid(alpha=0.3)

plt.tight_layout()
plt.show()

"""**Save Random Forest & XGBoost best models (.pkl)**"""

import joblib
import numpy as np

# Save Random Forest (with FE)
joblib.dump({
    'model': rf_fe,
    'feature_cols': rf_feature_cols,
    'name': 'Random Forest (FE, tuned)',
}, 'rf_model.pkl')

# Save XGBoost (with FE) – replace `xgb_fe` with your XGBoost variable name if different
joblib.dump({
    'model': xgb_fe,
    'feature_cols': feature_cols_fe,
    'name': 'XGBoost (FE, tuned)',
}, 'xgb_model.pkl')

print("Saved files:")
print(" rf_model.pkl ->", type(rf_fe).__name__, "with", len(rf_feature_cols), "features")
print(" xgb_model.pkl ->", type(xgb_fe).__name__, "with", len(feature_cols_fe), "features")

# Sanity check: reload both and confirm predictions match
rf_loaded = joblib.load('rf_model.pkl')
xgb_loaded = joblib.load('xgb_model.pkl')

assert np.array_equal(rf_loaded['model'].predict(X_val_rf), rf_fe.predict(X_val_rf))
assert np.array_equal(xgb_loaded['model'].predict(X_val_fe), xgb_fe.predict(X_val_fe))

```

```

"""***MLP**

**( WITHOUT ) Feature Engineering**
"""

# Import libraries
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score, log_loss
import time

# Build baseline MLP (no feature engineering)
MLPmodel = MLPClassifier(
    hidden_layer_sizes=( 64, 32), # 2 hidden layers
    activation='relu', # activation function
    max_iter=500, # max epochs
    early_stopping=True, # stop if no improvement
    alpha=0.05, # L2 regularization
    n_iter_no_change=10, # patience
    random_state=42
)

# Train and measure time
start = time.time()
MLPmodel.fit(X_train, y_train)
train_time = time.time() - start

# Predictions
train_preds3 = MLPmodel.predict(X_train)
val_preds3 = MLPmodel.predict(X_val)

# Probabilities for log-loss
train_proba3 = MLPmodel.predict_proba(X_train)
val_proba3 = MLPmodel.predict_proba(X_val)

# Calculate metrics
train_acc = accuracy_score(y_train, train_preds3)
val_acc = accuracy_score(y_val, val_preds3)
train_loss = log_loss(y_train, train_proba3)
val_loss = log_loss(y_val, val_proba3)

# Print results
print("=" * 55)
print("Baseline MLP (no feature engineering)")
print("=" * 55)
print(f"Train Accuracy : {train_acc:.4f}")
print(f"Val Accuracy : {val_acc:.4f}")
print(f"Accuracy Gap : {(train_acc - val_acc)*100:.2f} pp")
print()
print(f"Train Log-Loss : {train_loss:.4f}")
print(f"Val Log-Loss : {val_loss:.4f}")
print(f"Loss Gap : {val_loss - train_loss:.4f}")
print()
print(f"Training time : {train_time:.2f}s")

"""***Feature Engineering***"""

def add_strategic_features(df):
    df = df.copy()

    # 8 possible winning lines (rows, columns, diagonals)
    lines = [
        [0,1,2], [3,4,5], [6,7,8],
        [0,3,6], [1,4,7], [2,5,8],
        [0,4,8], [2,4,6]
    ]

```

```

# Convert board to numpy array for speed
board = df[['p1','p2','p3','p4','p5','p6','p7','p8','p9']].values

# Initialize feature lists
my_two_in_row = []
opp_two_in_row = []
my_fork = []
opp_fork = []
center_free = []
my_corners = []
empty_cells = []

# Four corner indices
corners = [0, 2, 6, 8]

for b in board:
    my_2, opp_2 = 0, 0

# Check each winning line for threats
for line in lines:
    vals = [b[i] for i in line]
    if vals.count(1) == 2 and vals.count(0) == 1:
        my_2 += 1 # player threatens to win
    if vals.count(-1) == 2 and vals.count(0) == 1:
        opp_2 += 1 # opponent threatens to win

my_two_in_row.append(my_2)
opp_two_in_row.append(opp_2)
my_fork.append(1 if my_2 >= 2 else 0) # player has a fork
opp_fork.append(1 if opp_2 >= 2 else 0) # opponent has a fork
center_free.append(1 if b[4] == 0 else 0) # center cell is empty
my_corners.append(sum(1 for c in corners if b[c] == 1)) # corners owned by player
empty_cells.append(list(b).count(0)) # remaining empty cells

# Add features to dataframe
df['my_two_in_row'] = my_two_in_row
df['opp_two_in_row'] = opp_two_in_row
df['my_fork'] = my_fork
df['opp_fork'] = opp_fork
df['center_free'] = center_free
df['my_corners'] = my_corners
df['empty_cells'] = empty_cells

return df

# Apply to train and validation sets
train_fe = add_strategic_features(train)
val_fe = add_strategic_features(val)

"""Train with features Engineering"""

# Define feature columns (original + engineered)
feature_cols_fe = ['p1','p2','p3','p4','p5','p6','p7','p8','p9','turn',
'my_two_in_row','opp_two_in_row',
'my_fork','opp_fork',
'center_free','my_corners','empty_cells']
# Split into X and y
X_train_fe = train_fe[feature_cols_fe]
y_train_fe = train_fe['label_move_cell']
X_val_fe = val_fe[feature_cols_fe]
y_val_fe = val_fe['label_move_cell']

# Feature count summary
print(f"Original features : {X_train.shape[1]}")
print(f"New features : {X_train_fe.shape[1]}")
print(f"Added features : {X_train_fe.shape[1] - X_train.shape[1]}")

```



```

# Build MLP with regularization to reduce overfitting
MLPmodel_fe = MLPClassifier(
    hidden_layer_sizes=(64, 32), # smaller network to reduce overfitting
    activation='relu',
    max_iter=500,
    early_stopping=True,
    alpha=0.05, # L2 regularization
    n_iter_no_change=10, # stop early if no improvement
    random_state=42
)

# Train the model
MLPmodel_fe.fit(X_train_fe, y_train_fe)

# Predictions
train_preds_fe = MLPmodel_fe.predict(X_train_fe)
val_preds_fe = MLPmodel_fe.predict(X_val_fe)

train_proba_fe = MLPmodel_fe.predict_proba(X_train_fe)
val_proba_fe = MLPmodel_fe.predict_proba(X_val_fe)

# Calculate accuracy and log-loss
train_acc_fe = accuracy_score(y_train_fe, train_preds_fe)
val_acc_fe = accuracy_score(y_val_fe, val_preds_fe)
train_loss_fe = log_loss(y_train_fe, train_proba_fe)
val_loss_fe = log_loss(y_val_fe, val_proba_fe)

# Print results
print("=" * 55)
print("MLP (with feature engineering)")
print("=" * 55)
print(f"Train Accuracy : {train_acc_fe:.4f}")
print(f"Val Accuracy : {val_acc_fe:.4f}")
print(f"Accuracy Gap : {(train_acc_fe - val_acc_fe)*100:.2f}")
print()
print(f"Train Log-Loss : {train_loss_fe:.4f}")
print(f"Val Log-Loss : {val_loss_fe:.4f}")
print(f"Loss Gap : {val_loss_fe - train_loss_fe:.4f}")

""" Save the final model"""

import joblib

# Save the final model
joblib.dump(MLPmodel_fe, 'MLP_final_model.pkl')
print("Model saved successfully!")

```